VALEMONT OS · MISE EN SERVICE

Le plan et le tuto de toutes les connexions

English version: docs/SETUP_GUIDE.en.md

Pour Théo et Pablo. Ce guide te dit **dans quel ordre** brancher chaque service, **où cliquer**, **quelle variable remplir** et **comment vérifier** que ça marche. Il est écrit à partir des documents du projet (ils font foi) : `.env.example`, `docs/DEPLOYMENT.md`, `docs/INTEGRATIONS.md`, `docs/INBOX.md`, `docs/TRACKER.md`, `docs/SECURITY.md`, `docs/ACCEPTANCE.md`. Les étapes chez Google, Vercel, Wise, Stripe, Meta et LinkedIn ont été vérifiées dans leurs documentations officielles en novembre 2026 (liens dans chaque section). Ces interfaces changent souvent : si un menu a un autre nom, cherche le mot-clé indiqué.

Trois règles avant de commencer

1. **Un secret (clé, jeton, mot de passe) ne se colle qu'à un seul endroit : Vercel › Settings › Environment Variables.** Jamais dans un chat (y compris avec Claude), jamais dans un e-mail, jamais dans le Google Sheet, jamais dans le code.
2. **Tous les comptes appartiennent à Valemont** (vos adresses Google, votre Business Manager Meta, votre compte Vercel). Le compte Claude qui a construit l'application n'est jamais utilisé sur vos vraies ressources.
3. **Après chaque changement de variable : Vercel › Deployments › ⋯ › Redeploy.** Une variable ajoutée n'est prise en compte qu'au déploiement suivant.

Dans l'application, tout se contrôle dans **Réglages › Intégrations** : une ligne par service (15 au total), avec son statut (*Connecté*, *Non configuré*, *Erreur*, *À renouveler*, *Import uniquement*, *En attente d'approbation*, *Démo, simulé*), les **noms** des variables (définie / manquante, jamais leur valeur) et les boutons **Tester la connexion**, **Synchroniser**, **Autoriser / Autoriser à nouveau** quand ils ont un sens.

---

1. Le plan

Coche au fur et à mesure. « Qui » est une proposition : l'important est que la personne ait les droits d'administration sur le compte concerné.

Phase 0 · Prérequis et comptes (les deux · 1 h)

- [ ] Choisir l'adresse de l'application, par exemple `os.<votre-domaine>` (elle servira partout : `APP_BASE_URL`).
- [ ] Vérifier que vous avez : un compte Google chacun (idéalement Google Workspace, même domaine), l'accès au dépôt GitHub `valemontops-boop/valemontops`, un compte Vercel, l'accès administrateur Wise Business, Stripe, Meta Business (Page Facebook + comptes Instagram professionnels), les Pages LinkedIn, GA4, Search Console, l'hébergement des sites (Netlify ou autre), et Make, Zapier ou n8n si vous voulez le relais de la boîte.
- [ ] Un ordinateur avec un terminal et **Node.js 22** installé (https://nodejs.org) pour la Phase 3 (on n'en a besoin qu'une fois, puis à chaque mise à jour qui ajoute des onglets).
- [ ] Un gestionnaire de mots de passe partagé (1Password, Bitwarden…) pour garder une copie des secrets générés.

Débloque : rien encore, mais évite d'être bloqué au milieu.

Phase 1 · Google : Sheets, Drive, connexion (Théo, ou le titulaire du compte Google admin · 1 h 30)

- [ ] Créer le projet Google Cloud et activer les 4 API (Sheets, Drive, Calendar, Gmail).
- [ ] Créer le **compte de service** et sa clé JSON → `GOOGLE_SERVICE_ACCOUNT_KEY_BASE64`.
- [ ] Créer le Google Sheet principal vide et le partager avec le compte de service → `SHEETS_SPREADSHEET_ID`.
- [ ] (Facultatif) Chacun crée son Sheet privé et le partage → `PRIVATE_SHEET_ID_THEO` / `PRIVATE_SHEET_ID_PABLO`.
- [ ] Partager le dossier Drive « Valemont OS » (Lecteur) et préparer le dossier des sauvegardes (Éditeur).
- [ ] Partager les agendas avec le compte de service.
- [ ] Écran de consentement OAuth + client OAuth → `GOOGLE_OAUTH_CLIENT_ID`, `GOOGLE_OAUTH_CLIENT_SECRET`.

Débloque : la base de données (Sheet), la connexion Google des deux fondateurs, Fichiers, Agenda.

Phase 2 · Mise en ligne sur Vercel (Théo ou Pablo · 45 min)

- [ ] Importer le dépôt GitHub dans Vercel, plan **Pro**, région Paris.

- [ ] Remplir les variables obligatoires (liste en §2.4), ajouter le domaine, redéployer.

Débloque : l'application en ligne en mode LIVE (sans les variables, elle affiche une page de configuration, jamais la démo).

Phase 3 · Premier démarrage (la personne qui a Node · 30 min, puis les deux · 10 min)

- [ ] `npm run setup:sheets` puis `npm run doctor:sheets` sur le vrai Sheet : tout en ✓.
- [ ] Théo se connecte, Pablo se connecte dans un autre navigateur ; un compte non fondateur est refusé.
- [ ] Réglages › Calendriers et Réglages › Espace de travail : coller les identifiants d'agendas et du dossier Drive.

Débloque : toute l'application de base (Ma journée, CRM, Appels, Tâches, Agenda, Fichiers).

Phase 4 · Argent : Wise et Stripe (celui qui administre la banque · 1 h)

- [ ] Wise : jeton API en lecture seule (+ identifiant de profil, + clé SCA facultative, + webhook).
- [ ] Stripe : clé restreinte en lecture + webhook.

Débloque : Finances (soldes, mouvements, ventes, frais, virements), MRR dans Finances et le Command Center, taux USD→EUR pour les paiements fournisseurs en dollars. L'application ne peut jamais déplacer d'argent.

Phase 5 · Mails (Gmail) et agenda (chacun pour soi · 10 min chacun, + délai Google éventuel)

- [ ] Vérifier que `gmail.send` est dans l'écran de consentement, puis chacun clique **Autoriser** sur la ligne Gmail.
- [ ] Vérifier que l'Agenda fusionne bien les agendas (l'agenda perso de l'autre apparaît en « Occupé »).

Débloque : envoi d'e-mails en un clic depuis Appels et Croissance › Messages (toujours après ton clic). ⏳ Avec une application Google « Externe », le consentement est redemandé souvent et la publication en production demande une **vérification Google** (voir §2.6).

Phase 6 · Site web et boîte « À répondre » (Pablo ou celui qui gère les sites · 1 h 30)

- [ ] Webhook signé des formulaires des sites FR/ES → `WEBSITE_WEBHOOK_SECRET`.
- [ ] Relais Make / Zapier / n8n (commentaires, notifications LinkedIn…) → `INBOX_WEBHOOK_SECRET`.

Débloque : chaque formulaire de site devient un prospect entrant + un élément « À répondre » ; les commentaires et messages arrivent dans Ma journée.

Phase 7 · Réseaux : Instagram, LinkedIn, GA4, Search Console (Théo ou Pablo · 2 h + délais Meta/LinkedIn)

- [ ] Instagram : app Meta, jeton (utilisateur système), identifiant du compte → statistiques automatiques.
- [ ] Instagram : webhook commentaires/messages → boîte « À répondre ». ⏳ **Examen de l'app par Meta** pour les comptes qui n'ont pas de rôle dans l'app.
- [ ] LinkedIn : imports d'exports (.xls Page, .xlsx créateur). ⏳ API Community Management : **demande d'accès** facultative, longue et non garantie.
- [ ] GA4 et Search Console : exports CSV importés chaque semaine.

Débloque : Croissance (canaux, coûts, entonnoirs), Studio contenu (chiffres par post), Croissance › SEO.

Phase 8 · Tracker perso (chacun pour soi · 20 min)

- [ ] URL + secret du webhook de ton app de suivi, jeton du flux.

Débloque : chacune de tes activités envoyée à ton tracker personnel, et un flux qu'il peut lire.

Phase 9 · Sauvegardes, tâches planifiées, test de restauration (Théo · 1 h)

- [ ] Dossier de sauvegarde (Drive partagé, ou copie Apps Script si compte Google personnel) → `BACKUP_DRIVE_FOLDER_ID`.
- [ ] `CRON_SECRET` (les deux tâches planifiées sont déjà déclarées dans `vercel.json`).

- [ ] Exercice de restauration dans un **nouveau** Sheet (ne jamais toucher au Sheet de production).

Débloque : copie quotidienne vérifiée du Sheet, capture Instagram toutes les 6 h, plan B en cas de pépin.

Phase 10 · Vérifications finales (les deux · 45 min)

- [ ] Réglages › Intégrations : chaque ligne configurée est **Connecté** après **Tester la connexion** ; GA4 et Search Console = *Import uniquement* (normal) ; LinkedIn = *En attente d'approbation* (normal).
- [ ] Réglages › Diagnostic : santé « prête », sauvegardes « vérifiée ».
- [ ] Dérouler la liste « Stage 4 gate: live validation » de `docs/ACCEPTANCE.md` et noter la date de chaque case dans `docs/STATUS.md` (rien n'est VERIFIED_LIVE avant ça).

---

2. Le tuto

2.1 Générer un secret (utile partout)

Plusieurs variables sont des secrets que **vous inventez** (pas fournis par un service). Génère-les sur ton ordinateur, colle-les dans Vercel et dans le gestionnaire de mots de passe :

Besoin	macOS / Linux (Terminal)	Windows (PowerShell 7)
`SESSION_SECRET`	`openssl rand -base64 48`	`[Convert]::ToBase64String([Security.Cryptography.RandomNumberGenerator]::GetBytes(48))`
tous les autres secrets/jetons (webhooks, cron, tracker)	`openssl rand -hex 32`	`-join ([Security.Cryptography.RandomNumberGenerator]::GetBytes(32) | % { $_.ToString("x2") })`

Un secret par usage : ne réutilise jamais le même pour deux variables.

2.2 Google Cloud : projet, API, compte de service

À quoi ça sert : le compte de service est le « robot » qui lit et écrit le Google Sheet (seule base de données), lit le dossier Drive partagé, lit les agendas et fait les sauvegardes.

Ce qu'il te faut : ton compte Google (de préférence Workspace), 30 minutes.

Étapes

1. Va sur https://console.cloud.google.com › sélecteur de projet › **Nouveau projet** › nom « Valemont OS ». (Doc : https://developers.google.com/workspace/guides/create-project)
2. **API et services › Bibliothèque** : active **Google Sheets API**, **Google Drive API**, **Google Calendar API**, **Gmail API**. (Doc : https://developers.google.com/workspace/guides/enable-apis)
3. **IAM et administration › Comptes de service › Créer un compte de service** : nom `valemont-os`, **aucun rôle** (ses droits viennent uniquement des partages que tu fais). Note son adresse : `valemont-os@<projet>.iam.gserviceaccount.com`. (Doc : https://cloud.google.com/iam/docs/service-accounts-create)
4. Ouvre le compte de service › onglet **Clés › Ajouter une clé › Créer une clé › JSON**. Un fichier se télécharge. (Doc : https://cloud.google.com/iam/docs/keys-create-delete)
5. Encode-le en base64 et copie-le dans le presse-papiers :
   - macOS : `base64 -i key.json | pbcopy`
   - Linux : `base64 -w0 key.json`

    - Windows : `[Convert]::ToBase64String([IO.File]::ReadAllBytes("key.json")) | Set-Clipboard`
6. Colle la valeur dans Vercel (Phase 2) sous `GOOGLE_SERVICE_ACCOUNT_KEY_BASE64`, puis **supprime le fichier** `key.json` (et vide la corbeille).

Variables : `GOOGLE_SERVICE_ACCOUNT_KEY_BASE64`.

Problèmes fréquents

- *« La création de clés de compte de service est désactivée »* : les organisations Google Cloud récentes bloquent les clés par défaut (règle `iam.disableServiceAccountKeyCreation`). L'administrateur Workspace doit lever cette règle **pour ce projet seulement** (IAM › Règles d'administration), créer la clé, puis la remettre.
- Clé perdue ou fuitée : crée une nouvelle clé, remplace la variable, redéploie, **puis** supprime l'ancienne clé.

2.3 Google Sheets (base de données) et Sheets privés

À quoi ça sert : tout ce que vous saisissez est lu et écrit ici. Les Sheets privés gardent les tâches privées et candidatures de chacun, invisibles pour l'autre.

Étapes

1. Sur https://sheets.google.com, crée un tableur **vide** « Valemont OS » (l'application ne le crée jamais).
2. **Partager** › colle l'adresse du compte de service › rôle **Éditeur** › décoche « Envoyer une notification ».
3. Copie l'identifiant dans l'URL : `https://docs.google.com/spreadsheets/d/<CET-IDENTIFIANT>/edit` → `SHEETS_SPREADSHEET_ID`.
4. (Facultatif) **Chacun** crée **depuis son propre compte** un tableur « Valemont OS – privé <prénom> », le partage avec le compte de service en **Éditeur** (et avec personne d'autre) → `PRIVATE_SHEET_ID_THEO`, `PRIVATE_SHEET_ID_PABLO`.
5. Les onglets et en-têtes sont créés en Phase 3 par `npm run setup:sheets`.

Variables : `SHEETS_SPREADSHEET_ID` (obligatoire), `PRIVATE_SHEET_ID_THEO`, `PRIVATE_SHEET_ID_PABLO` (facultatives).

Vérifier : Réglages › Intégrations › Google Sheets › **Tester la connexion** ; `npm run doctor:sheets` tout en ✓.

Problèmes fréquents

- *Permission refusée* : le partage a été fait avec une autre adresse, ou en Lecteur au lieu d'Éditeur.
- Ne renomme pas et ne supprime pas les onglets/colonnes à la main : l'application retrouve les colonnes par leur nom. Ajouter des lignes de mise en forme ou des formules dans le Sheet casse

la lecture (Diagnostic le signale).

2.4 Google Sign-In (connexion des deux fondateurs)

À quoi ça sert : seules vos deux adresses Google peuvent se connecter. Le même client OAuth sert aussi aux autorisations Drive (`drive.file`) et Gmail (`gmail.send`), demandées plus tard, une par une.

Étapes (le menu s'appelle maintenant **Google Auth Platform** dans la console)

1. **Google Auth Platform › Branding** : nom « Valemont OS », adresse d'assistance = l'adresse d'un fondateur, domaine autorisé = votre domaine. (Doc : https://developers.google.com/workspace/guides/configure-oauth-consent)
2. **Audience** :
   - si vous êtes **tous les deux dans le même Google Workspace** : type **Interne** (le plus simple : pas de vérification, pas d'utilisateurs de test) ;
   - sinon : type **Externe**, statut **Test**, et ajoute vos deux adresses dans **Utilisateurs de test**.
3. **Accès aux données › Ajouter ou supprimer des champs d'application** : `openid` , `.../auth/userinfo.email` , `.../auth/userinfo.profile` , `.../auth/drive.file` , `.../auth/gmail.send` . (Les deux derniers ne sont demandés qu'au moment où tu t'en sers : c'est l'autorisation incrémentale, https://developers.google.com/identity/protocols/oauth2/web-server#incrementalAuth)
4. **Clients › Créer un client** › type **Application Web** › nom « Valemont OS ». **URI de redirection autorisés** (remplace par ton adresse, sans `/` final) :
   - `https://os.<domaine>/api/auth/google/callback`
   - `https://os.<domaine>/api/auth/google/drive/callback`
   - `https://os.<domaine>/api/auth/google/gmail/callback`
5. Copie l'**ID client** → `GOOGLE_OAUTH_CLIENT_ID` et le **code secret** → `GOOGLE_OAUTH_CLIENT_SECRET` (directement dans Vercel).

Variables : `GOOGLE_OAUTH_CLIENT_ID` , `GOOGLE_OAUTH_CLIENT_SECRET` , `FOUNDER_THEO_EMAIL` , `FOUNDER_PABLO_EMAIL` , `FOUNDER_THEO_TIMEZONE` (`Europe/Paris`), `FOUNDER_PABLO_TIMEZONE` (`Europe/Madrid`), `APP_BASE_URL` .

Vérifier : ouvrir l'application, « Se connecter avec Google » : Théo et Pablo entrent, une troisième adresse est refusée.

Problèmes fréquents

- `redirect_uri_mismatch` : l'URI enregistrée ne correspond pas exactement à `APP_BASE_URL` (https, pas de `/` final, même sous-domaine). Si tu changes de domaine, ajoute les 3 nouvelles URI.
- *« Accès bloqué : l'application n'a pas terminé la procédure de validation »* : en Externe/Test, l'adresse n'est pas dans les utilisateurs de test.

2.5 Google Drive et Google Calendar

À quoi ça sert : Fichiers liste le dossier « Valemont OS » ; l'envoi d'un fichier se fait avec **ta** permission `drive.file` (le fichier t'appartient). L'Agenda fusionne l'agenda partagé « Valemont » et vos agendas ; l'agenda perso de l'autre n'apparaît qu'en « Occupé ». Les points fondateurs sont écrits dans l'agenda partagé.

Étapes Drive

1. Dans Google Drive, dossier « Valemont OS » › **Partager** › compte de service en **Lecteur**.
2. Copie l'identifiant dans l'URL `https://drive.google.com/drive/folders/<IDENTIFIANT>` et colle-le dans l'application : **Réglages › Espace de travail › Identifiant du dossier Drive** (pas une variable d'environnement).

Étapes Calendar (Doc : https://support.google.com/calendar/answer/37082)

1. Google Agenda › ⚙ **Paramètres** › choisis l'agenda partagé « Valemont » › **Partager avec des personnes ou des groupes spécifiques** › ajoute le compte de service avec **Apporter des modifications aux événements**.
2. Chacun, pour son agenda personnel : même chemin, mais avec **Voir uniquement les disponibilités (masquer les détails)**.
3. Dans chaque agenda › **Intégrer l'agenda › ID de l'agenda** : copie-le dans **Réglages › Calendriers**.

Variables : aucune nouvelle (`GOOGLE_SERVICE_ACCOUNT_KEY_BASE64` suffit).

Vérifier : Réglages › Intégrations › Google Drive et Google Calendar › **Tester la connexion** ; ouvrir Fichiers et Agenda. Premier envoi de fichier : Google te demande la permission Drive une fois (elle dure environ 1 h, puis un clic sur **Autoriser à nouveau**).

Problèmes fréquents : agenda vide = mauvais ID ou partage oublié ; fichier invisible = il n'est pas dans le dossier partagé.

2.6 Gmail (envoi seulement)

À quoi ça sert : les e-mails préparés par l'application (après un appel, relances, Croissance › Messages) partent de **ton** Gmail **quand tu cliques**. L'application ne lit jamais ta boîte de réception. Sans Gmail connecté, elle ouvre un brouillon Gmail pré-rempli : rien n'est bloqué.

Étapes

1. Vérifie (Phase 1, étape 3 du §2.4) que la Gmail API est activée, que `gmail.send` est dans l'Accès aux données et que l'URI `/api/auth/google/gmail/callback` est enregistrée.
2. Chacun, dans l'application : **Réglages › Intégrations › Gmail › Autoriser** (ou « Connecter Gmail » dans Appels). Google affiche l'écran de consentement : accepte.

Variables : aucune nouvelle (même client OAuth).

Vérifier : la ligne Gmail montre l'autorisation active dans ce navigateur. Il n'y a **pas** de « Tester la connexion » pour Gmail (un test enverrait un vrai mail) : envoie-toi un e-mail de test depuis une fiche contact à ton adresse.

⏳ **Délais et limites Google** (Doc : https://developers.google.com/workspace/gmail/api/auth/scopes, https://support.google.com/cloud/answer/13463073)

- `gmail.send` est un champ d'application **sensible**.
- Application **Interne** (Workspace) : aucune vérification, rien à attendre. C'est le choix recommandé.
- Application **Externe en Test** : fonctionne pour vos deux adresses (écran « application non validée » à accepter), mais Google redemande le consentement souvent (au moins tous les 7 jours).
- Passer une application **Externe en production** avec `gmail.send` demande la **vérification Google** (vidéo de démonstration, politique de confidentialité, domaine vérifié) : compte **plusieurs jours à quelques semaines**.

2.7 Vercel : mise en ligne

À quoi ça sert : héberger l'application, garder les secrets, lancer les deux tâches planifiées.

Étapes

1. https://vercel.com › **Add New › Project** › **Import Git Repository** › autorise l'app GitHub de Vercel sur `valemontops-boop/valemontops` › Framework : Next.js (détecté), build par défaut. (Doc : https://vercel.com/docs/getting-started-with-vercel/import)
2. **Plan Pro** : le plan Hobby est réservé à un usage personnel non commercial et ne permet pas une tâche toutes les 6 h. Un seul membre payant suffit. (Doc : https://vercel.com/docs/limits/fair-use-guidelines, https://vercel.com/docs/cron-jobs/usage-and-pricing)
3. **Settings › Functions** : région **Paris (cdg1)** ; laisse le runtime Node.js et Fluid Compute (par défaut).
4. **Settings › Environment Variables** : ajoute chaque variable, environnement **Production**, et coche **Sensitive** pour tout secret (valeur ensuite illisible, même pour vous). (Doc : https://vercel.com/docs/environment-variables). Minimum pour démarrer : `APP_MODE=live`, `SESSION_SECRET`, `APP_BASE_URL`, `FOUNDER_THEO_EMAIL`, `FOUNDER_PABLO_EMAIL`, `FOUNDER_THEO_TIMEZONE`, `FOUNDER_PABLO_TIMEZONE`, `GOOGLE_OAUTH_CLIENT_ID`, `GOOGLE_OAUTH_CLIENT_SECRET`, `SHEETS_SPREADSHEET_ID`, `GOOGLE_SERVICE_ACCOUNT_KEY_BASE64`, `CRON_SECRET`, `LOG_LEVEL=info`.
5. **Settings › Domains** › ajoute `os.<domaine>` › crée chez ton registrar l'enregistrement DNS indiqué (CNAME) › attends le certificat. Mets `APP_BASE_URL=https://os.<domaine>` (sans `/` final). (Doc : https://vercel.com/docs/domains/working-with-domains/add-a-domain)

6. **Deployments › ⋯ › Redeploy**.
7. Tâches planifiées : rien à créer, `vercel.json` déclare la sauvegarde quotidienne (02:17 UTC) et la capture Instagram (toutes les 6 h). Vercel les appelle avec `Authorization: Bearer <CRON_SECRET>`. Elles ne tournent que sur le déploiement de production. Visible dans **Settings › Cron Jobs**. (Doc : https://vercel.com/docs/cron-jobs/manage-cron-jobs#securing-cron-jobs)

Ne jamais mettre en production : `SHEETS_API_BASE_URL`, `GOOGLE_API_BASE_URL`, `WISE_API_BASE_URL`, `STRIPE_API_BASE_URL`, `DEMO_BACKEND`, `FAKE_SHEETS_URL`, `FAKE_SHEETS_SPREADSHEET_ID`, `DEMO_ACCESS_CODE` (outils de test et de démo). `INSTAGRAM_API_BASE_URL` seulement dans le cas Instagram Login (§2.12).

Vérifier : `https://os.<domaine>/api/health` répond `{"status":"ok"}` ou `"degraded"` (`fail` = une variable obligatoire manque ou le Sheet est inaccessible).

Problèmes fréquents : page « configuration requise » = une variable obligatoire manque (le nom est affiché) ; variable ajoutée mais rien ne change = pas redéployé.

2.8 Premier démarrage (Phase 3)

Ces commandes préparent le Sheet. Elles se lancent **une fois** depuis un ordinateur, avec vos propres accès.

1. Récupère le code : `git clone https://github.com/valemontops-boop/valemontops.git` puis `cd valemontops` et `npm ci`.
2. Copie `.env.example` en `.env.local` et remplis au moins `APP_MODE=live`, `SHEETS_SPREADSHEET_ID`, `GOOGLE_SERVICE_ACCOUNT_KEY_BASE64` (les mêmes valeurs que dans Vercel). Ce fichier n'est jamais envoyé sur GitHub.
3. `npm run setup:sheets` : crée les onglets et en-têtes, fige les en-têtes, ajoute une protection d'avertissement. Sans danger à relancer : il n'efface jamais rien. À relancer après chaque mise à jour qui ajoute des colonnes.
4. `npm run doctor:sheets` : chaque contrôle doit être ✓ (partage, en-têtes, doublons, schéma).
5. **Supprime** `.env.local` une fois fini (ou garde-le chiffré) : il contient la clé du compte de service.
6. Dans l'application : Théo se connecte, Pablo se connecte dans un autre navigateur. Remplissez Réglages › Calendriers et Réglages › Espace de travail. Ouvrez **Réglages › Diagnostic** : santé « prête ».

2.9 Wise (banque, lecture seule)

À quoi ça sert : page Finances et trésorerie du Command Center : soldes, relevés, mouvements en direct par webhook. Le même jeton en lecture seule donne aussi les **taux USD→EUR de Wise** utilisés pour les **paiements fournisseurs en dollars** (fonction en cours d'ajout). **Le jeton ne peut jamais déplacer d'argent.**

Ce qu'il te faut : accès administrateur au compte **Wise Business**, ton téléphone (double authentification).

Étapes (Doc : https://docs.wise.com/guides/developer/auth-and-security/personal-api-token)

1. wise.com › compte Business › **Paramètres › API tokens** (selon la version : *Connect and manage apps › API tokens* ou *Integrations and Tools › API tokens*) › **Add new token** › nom « Valemont OS » › accès **Read only** › valide la double authentification. Le jeton n'est affiché **qu'une fois** : colle-le directement dans Vercel → `WISE_API_TOKEN`.
2. (Facultatif) `WISE_PROFILE_ID` : sans lui, l'application prend le premier profil Business du jeton. Pour le connaître, dans un terminal (le jeton n'apparaît pas à l'écran) : `read -s WISE_TOKEN && curl -s -H "Authorization: Bearer $WISE_TOKEN" https://api.wise.com/v2/profiles` → l'`id` du profil de type `BUSINESS`.
3. (Facultatif) **Clé SCA** pour lire les relevés quand Wise exige une authentification forte :
   - `openssl genrsa -out wise-private.pem 2048` puis `openssl rsa -pubout -in wise-private.pem -out wise-public.pem` ;
   - Wise › API tokens › **Manage public keys** › ajoute le contenu de `wise-public.pem` ;
   - `base64 -i wise-private.pem` (macOS) → `WISE_SCA_PRIVATE_KEY_BASE64`, puis supprime les fichiers `.pem`.
   - Attention : dans l'UE/au Royaume-Uni (DSP2), Wise peut refuser l'accès aux relevés via un jeton personnel, même avec la clé. Dans ce cas les soldes et le webhook fonctionnent quand même et l'historique vient d'un **export CSV de relevé Wise** : Finances › **Importer un CSV Wise**.
4. **Webhook des mouvements** (Doc : https://docs.wise.com/guides/developer/webhooks/subscribe-to-webhooks) :
   - URL : `https://os.<domaine>/api/webhooks/wise`, événement `balances#update` ;
   - dans Wise › Paramètres › **Webhooks** (outils développeurs) si le menu existe sur ton compte ; sinon par l'API : `curl -X POST -H "Authorization: Bearer $WISE_TOKEN" -H "Content-Type: application/json" https://api.wise.com/v3/profiles/<PROFILE_ID>/subscriptions -d '{"name":"Valemont OS","trigger_on":"balances#update","delivery":{"version":"2.0.0","url":"https://os.<domaine>/api/webhooks/wise"}}'` (si Wise refuse avec un jeton en lecture seule, crée l'abonnement depuis l'interface, ou saute cette étape : le bouton **Synchroniser** rattrape les mouvements) ;
   - copie la **clé publique de production** des webhooks Wise (page « Event handling » de la doc : https://docs.wise.com/guides/developer/webhooks/event-handling), enregistre-la dans `wise-webhook.pem`, `base64 -i wise-webhook.pem` → `WISE_WEBHOOK_PUBLIC_KEY_BASE64`.

Variables : `WISE_API_TOKEN` (obligatoire pour Wise), `WISE_PROFILE_ID`, `WISE_SCA_PRIVATE_KEY_BASE64`, `WISE_WEBHOOK_PUBLIC_KEY_BASE64` (facultatives).

Vérifier : Réglages › Intégrations › Wise › **Tester la connexion** (lit les soldes) puis **Synchroniser** (relevés). Les soldes apparaissent dans Finances. Un crédit classé automatiquement en revenu reste « à

vérifier » tant qu'un fondateur ne l'a pas confirmé.

Problèmes fréquents : 401 = jeton révoqué ou mal collé (espace en trop) ; « SCA requise » = clé SCA absente ou refusée → import CSV ; mauvais profil = renseigne `WISE_PROFILE_ID`.

2.10 Stripe (lecture seule)

À quoi ça sert : solde Stripe, ventes, frais, virements, MRR (Finances et Command Center). La clé ne peut ni encaisser ni rembourser. Les virements Stripe → Wise sont des transferts internes : une vente n'est jamais comptée deux fois.

Étapes (Doc : https://docs.stripe.com/keys/restricted-api-keys, https://docs.stripe.com/webhooks)

1. Dashboard Stripe (mode **live**) › **Développeurs › Clés API › Créer une clé restreinte** › nom « Valemont OS ». Mets **Lecture** sur : *Balance*, *Balance transactions*, *Subscriptions*, *Customers*. Tout le reste : **Aucun**. Copie la clé (`rk_live_…`) dans Vercel → `STRIPE_API_KEY`.
2. **Développeurs › Webhooks** (dans Workbench : *Destinations d'événements*) › **Ajouter une destination** › URL `https://os.<domaine>/api/webhooks/stripe` › événements : `charge.*`, `payout.*`, `customer.subscription.*`, `invoice.paid`, `balance.available`.
3. Ouvre la destination › **Secret de signature** › Révéler (`whsec_…`) → `STRIPE_WEBHOOK_SECRET`.

Variables : `STRIPE_API_KEY` (obligatoire pour Stripe), `STRIPE_WEBHOOK_SECRET` (facultative mais recommandée).

Vérifier : Tester la connexion (lit le solde), **Synchroniser** ; dans Stripe, *Envoyer un événement de test* sur la destination doit répondre 200.

Problèmes fréquents : 403 « permission manquante » = une ressource n'est pas en Lecture (le message dit laquelle) ; 400 sur le webhook = secret d'une autre destination ou du mode test.

2.11 Site web : formulaires → webhook signé

À quoi ça sert : chaque formulaire (contact, prospect, réservation) des sites FR/ES crée un prospect entrant (propriétaire Théo pour FR, Pablo pour ES) et un élément « À répondre ». Un contact désinscrit est bloqué : rien n'est créé.

Étapes

1. Génère `WEBSITE_WEBHOOK_SECRET` (`openssl rand -hex 32` , 32 caractères minimum) et mets-le dans Vercel.
2. Mets **le même secret** dans l'hébergement du site (Netlify › Site configuration › Environment variables) avec `VALEMONT_WEBHOOK_URL=https://os.<domaine>/api/webhooks/website`.

3. Netlify Forms : le webhook sortant de Netlify ne sait pas signer, il faut une fonction **submission-created** (code prêt à copier dans `docs/INTEGRATIONS.md` › « Netlify Forms »). Doc : https://docs.netlify.com/functions/trigger-on-events/. Autre hébergeur : n'importe quel code côté serveur (Worker Cloudflare, Apps Script…) qui signe de la même façon (`docs/INTEGRATIONS.md` › « Generic form handler »). **Le secret ne doit jamais être dans le code du navigateur.**
4. Dans le formulaire : champ piège `bot-field`, champs cachés `lang`, `page`, UTM, `consent_version`, et une case de consentement dont la phrase est envoyée en `consent_text`.

Variables : `WEBSITE_WEBHOOK_SECRET` (+ `VALEMONT_WEBHOOK_URL` côté site, pas dans Vercel).

Vérifier : Tester la connexion (vérifie que le secret est présent et assez long), puis envoie un vrai formulaire de test : il apparaît dans Ma journée › À répondre.

Problèmes fréquents : 401 `stale_timestamp` = horloge du serveur décalée de plus de 5 min ; 401 `bad_signature` = secrets différents ou signature calculée sur un JSON reformaté (il faut signer le texte exact envoyé).

2.12 Boîte « À répondre » : relais Make / Zapier / n8n et Instagram

À quoi ça sert : rassembler ce qui attend une réponse (commentaires, DM, notifications LinkedIn…) avec une réponse suggérée dans ta langue. **Rien n'est jamais envoyé automatiquement** : « Copier une réponse » ouvre la conversation et tu colles toi-même. Les réponses e-mail/WhatsApp marquées et les prospects du site arrivent déjà sans rien brancher.

A. Relais (Make, Zapier ou n8n)

1. Génère `INBOX_WEBHOOK_SECRET` et mets-le dans Vercel **et** dans le relais (variable / connexion protégée).
2. Déclencheur au choix : « Instagram for Business › Watch comments », boîte mail du relais (Make *Mailhook*, *Email by Zapier*) alimentée par un **filtre Gmail** qui transfère les notifications LinkedIn, etc.
3. Action : HTTP POST vers `https://os.<domaine>/api/inbox/webhook`, corps JSON (format dans `docs/INBOX.md`), en-tête `X-Valemont-Signature: sha256=<HMAC-SHA256 du corps exact avec le secret>` : Make = fonction `sha256(body; "hex"; secret)` en mode HMAC ; Zapier = étape *Code by Zapier* ; n8n = nœud *Crypto* (HMAC, SHA256, hex). `external_id` doit être stable (c'est l'anti-doublon).

B. Instagram via l'app Meta (voir aussi §2.13 pour l'app elle-même)

1. Dans l'app Meta › **Webhooks** (ou Instagram › *API setup* › *Configure webhooks*) : URL de rappel `https://os.<domaine>/api/inbox/meta`, jeton de vérification = une valeur que tu génères

→ `META_VERIFY_TOKEN`. (Doc : https://developers.facebook.com/docs/instagram-platform/webhooks)
2. Abonne les champs `comments` et `messages` pour les deux comptes (Valemont, 30/30).
3. **Paramètres de l'app › Général › Clé secrète** → `META_APP_SECRET`.
4. (Facultatif) Nommer les comptes : réglage `instagram_account_labels` (voir `docs/INBOX.md`).
5. ⏳ Permissions `instagram_business_manage_comments`, `instagram_business_manage_messages` (ou équivalents `instagram_manage_comments` / `instagram_manage_messages`) : pour recevoir les événements de comptes qui n'ont pas de rôle dans l'app, Meta exige l'**App Review** (quelques jours à quelques semaines, vidéo de démonstration, politique de confidentialité). En attendant, utilisez le relais Make « Watch comments ». (Doc : https://developers.facebook.com/docs/resp-plat-initiatives/individual-processes/app-review)

Variables : `INBOX_WEBHOOK_SECRET`, `META_APP_SECRET`, `META_VERIFY_TOKEN`.

Vérifier : Réglages › Intégrations › Boîte À répondre › **Tester la connexion** ; puis un commentaire de test sur Instagram (ou un appel du relais) apparaît dans Ma journée › À répondre.

Problèmes fréquents : Meta n'arrive pas à « vérifier et enregistrer » l'URL = `META_VERIFY_TOKEN` différent ou pas redéployé ; 401 du relais = signature calculée sur un autre texte que le corps envoyé.

2.13 Instagram : statistiques (lecture seule) et capture toutes les 6 h

À quoi ça sert : abonnés, vues, portée, visites de profil, clics vers le site, et chiffres de chaque post/Reel à 24 h, 72 h, 7 j, 30 j, stories toutes les 6 h (Studio contenu, Croissance). Sans jeton : saisie à la main, marquée « saisi à la main ».

Ce qu'il te faut : chaque compte Instagram en **compte professionnel** (Business ou Créateur) **relié à une Page Facebook** (Doc : https://developers.facebook.com/docs/instagram-platform/overview), le tout dans le **portefeuille business Meta de Valemont**.

Étapes (méthode recommandée : Facebook Login + utilisateur système, jeton sans expiration)

1. https://developers.facebook.com › **Mes apps › Créer une app** › type Business, rattachée au portefeuille Valemont (jamais un compte personnel tiers). Ajoute le produit **Instagram** (API avec connexion Facebook). (Doc : https://developers.facebook.com/docs/instagram-platform/instagram-api-with-facebook-login/)
2. **Paramètres business** (business.facebook.com/settings) › **Utilisateurs › Utilisateurs système › Ajouter** (rôle admin) › **Attribuer des éléments** : l'app, la Page et les comptes Instagram. (Doc : https://developers.facebook.com/docs/marketing-api/system-users/)

3. **Générer un jeton** › app Valemont OS › expiration **Jamais** › autorisations `instagram_basic`, `instagram_manage_insights`, `pages_show_list`, `pages_read_engagement` → `INSTAGRAM_ACCESS_TOKEN`.
4. Identifiant du compte Instagram : dans l'Explorateur de l'API Graph, `me/accounts?fields=instagram_business_account` → le champ `instagram_business_account.id` → `INSTAGRAM_BUSINESS_ACCOUNT_ID`.
5. Un compte : `INSTAGRAM_ACCOUNT_KEY=ig-valemont` (ou `ig-thirty`). **Deux comptes** : à la place, `INSTAGRAM_ACCOUNTS_JSON` (format exact dans `.env.example`).
6. Accès Standard sur vos propres comptes : **pas d'App Review nécessaire** pour les statistiques.

Méthode de secours (Instagram Login, jeton de 60 jours) : permissions `instagram_business_basic` + `instagram_business_manage_insights`, `INSTAGRAM_API_BASE_URL=https://graph.instagram.com/v21.0`, `INSTAGRAM_TOKEN_EXPIRES_AT=AAAA-MM-JJ`. L'application affiche les jours restants et crée une tâche 14 jours avant ; renouvellement : https://developers.facebook.com/docs/instagram-platform/reference/refresh_access_token, puis remplace la variable et redéploie.

Capture toutes les 6 h : déjà planifiée par Vercel Cron (`CRON_SECRET`). `SNAPSHOT_JOB_SECRET` / `SNAPSHOT_JOB_TOKEN` ne servent que si un autre planificateur l'appelle (GitHub Actions, Apps Script…).

Vérifier : Tester la connexion (lit le profil), **Synchroniser** ; la ligne « Tâche de capture » montre la dernière exécution ; Studio contenu affiche les chiffres.

Problèmes fréquents : pas d'`instagram_business_account` = compte non professionnel ou non relié à la Page ; statistiques de story « non disponible » sous 5 spectateurs (c'est normal, jamais 0) ; démographie absente sous 100 abonnés.

2.14 LinkedIn (imports, et API plus tard)

À quoi ça sert : impressions, abonnés et pays des Pages Valemont / 30/30 et de vos profils créateurs, dans Croissance. La prospection LinkedIn reste manuelle (journal + ouverture du profil) : pas de scraping, pas de message automatique.

Maintenant : les exports

1. Page LinkedIn (en tant qu'admin) › **Analytics** › onglet (Contenu, Visiteurs, Abonnés) › **Exporter** › période › fichier **.xls**.
2. Profil créateur : **Analytics** de ton profil › **Exporter** › fichier **.xlsx**.
3. Dans l'application : Croissance › **Ajouter des chiffres** › **Importer un export LinkedIn**. Réimporter le même fichier n'ajoute rien en double. Une fois par semaine suffit.

Plus tard (⏳ facultatif) : API Community Management

- Créer une **nouvelle** app LinkedIn Developers dédiée (aucun autre produit), reliée à la **Page vérifiée** de Valemont, demander l'accès *Development tier*, puis le *Standard tier* avec une vidéo de chaque usage. Examen long, refus possible (il faut alors une nouvelle app). L'adaptateur ne sera construit qu'après approbation. (Doc : https://learn.microsoft.com/en-us/linkedin/marketing/community-management/community-management-overview, https://learn.microsoft.com/en-us/linkedin/marketing/increasing-access)

Variables : aucune. **Statut attendu :** *En attente d'approbation* (normal).

2.15 GA4 et Search Console (imports CSV)

À quoi ça sert : visites du site et prospects par source (dont les moteurs d'IA) dans Croissance ; requêtes et pages (clics, impressions, position) dans Croissance › SEO et l'optimiseur du Studio contenu.

GA4 (Doc : https://support.google.com/analytics/answer/9317657)

1. analytics.google.com › **Rapports** › Acquisition › Acquisition de trafic › période (ex. 7 derniers jours).
2. **Partager ce rapport** (en haut à droite) › **Télécharger le fichier** › **CSV**.
3. Croissance › **Ajouter des chiffres** › importer le CSV GA4.

Search Console (Doc : https://support.google.com/webmasters/answer/7576553)

1. search.google.com/search-console › propriété du site › **Performances** › période.
2. **Exporter** › **Télécharger le fichier CSV**.
3. Croissance › SEO › **Ajouter des chiffres Search Console**.

Variables : aucune. **Statut attendu :** *Import uniquement* avec la date du dernier import. Routine conseillée : le lundi, les deux exports.

2.16 Mon tracker (ton app de suivi personnelle)

À quoi ça sert : chacune de tes activités (appels, e-mails, visites…) est envoyée à ton tracker personnel, signée ; ton tracker peut aussi lire tes activités via un flux. Seulement en LIVE. Chacun configure **les siennes** : l'autre ne voit pas tes variables.

Étapes

1. Dans ton app de suivi, trouve son **URL de webhook entrant** (https obligatoire) → `TRACKER_WEBHOOK_URL_THEO` (ou `_PABLO`).
2. Génère un secret (16+ caractères) → `TRACKER_WEBHOOK_SECRET_THEO` (ou `_PABLO`), et donne-le à ton tracker pour qu'il vérifie l'en-tête `X-Valemont-Signature` (code de vérification dans `docs/TRACKER.md`).

3. (Facultatif) Flux : génère un jeton de 24+ caractères → `TRACKER_FEED_TOKEN_THEO` (ou `_PABLO`) ; ton tracker appelle `GET https://os.<domaine>/api/tracker/feed?from=AAAA-MM-JJ&to=AAAA-MM-JJ` avec `Authorization: Bearer <jeton>`.
4. (Facultatif) `TRACKER_DAILY_SUMMARY_THEO=1` pour envoyer aussi le résumé du jour à l'enregistrement du bilan.

Vérifier : Réglages › **Mon tracker** › **Envoyer un événement de test** (ou **Tester la connexion** dans Intégrations) ; « Envoyer cette période » rattrape un trou sans doublon.

Problèmes fréquents : URL en `http://` refusée ; le tracker doit répondre 2xx en moins de 5 s.

2.17 Sauvegardes, tâches planifiées, restauration

À quoi ça sert : une copie Google Sheet quotidienne, vérifiée, gardée 30 jours ; une restauration testée.

Étapes

1. Dossier de sauvegarde :
   - **Google Workspace** : crée un **Drive partagé** « Valemont OS – sauvegardes », ajoute le compte de service en **Gestionnaire de contenu** ;
   - **compte Google personnel** : un compte de service n'a pas d'espace de stockage. Crée un dossier normal, partage-le en Éditeur, et fais la copie avec le petit **Apps Script** à toi (code et déclencheur quotidien dans `docs/DEPLOYMENT.md` › 6. Backups).
2. Identifiant du dossier (URL) → `BACKUP_DRIVE_FOLDER_ID`. `BACKUP_RETENTION_DAYS=30` (1 à 365).
3. `CRON_SECRET` (24+ caractères) dans Vercel : Vercel Cron lance la sauvegarde à 02:17 UTC. `BACKUP_JOB_SECRET` / `BACKUP_JOB_TOKEN` seulement pour un autre planificateur.
4. **Exercice de restauration** (une fois, puis chaque trimestre) : crée un Sheet **vide** « Valemont OS (restauré AAAA-MM-JJ) », partage-le avec le compte de service, puis depuis l'ordinateur de la Phase 3 : `npm run restore:sheets -- --from-sheet <id de la copie> --to <id du nouveau Sheet>` → ✓, puis `SHEETS_SPREADSHEET_ID=<nouvel id> npm run doctor:sheets` → ✓. **Ne bascule pas la production** pour un exercice. Le script refuse le Sheet de production et tout Sheet non vide.
5. (Facultatif) Un moniteur de disponibilité (UptimeRobot, Better Stack…) sur `https://os.<domaine>/api/health`, une fois par minute au plus.

Vérifier : Réglages › Diagnostic › **Sauvegardes** : « Valemont OS backup » présente et **vérifiée** ; bouton **Vérifier** pour comparer à la base actuelle.

Problèmes fréquents : `storageQuotaExceeded` = compte personnel sans Drive partagé → solution Apps Script.

2.18 Vérifications finales (Phase 10)

1. Réglages › Intégrations : clique **Tester la connexion** sur chaque ligne configurée → **Connecté**.
2. Réglages › Diagnostic : santé « prête », sauvegarde vérifiée.
3. Dérouler la checklist « Stage 4 gate: live validation » de `docs/ACCEPTANCE.md` à deux (connexion des deux, appel vu par l'autre en moins d'une minute, conflit d'édition, `+33…` intact dans le Sheet, agenda, fichiers, imports LinkedIn, sauvegarde, restauration, `/api/health`, en-têtes de sécurité).
4. Noter le résultat daté dans `docs/STATUS.md`. Une intégration n'est **VERIFIED_LIVE** qu'après ce test réel.

---

3. Toutes les variables

À coller dans **Vercel › Settings › Environment Variables** (Production, *Sensitive* pour les secrets), puis **Redeploy**. « Vous » = une valeur que vous générez vous-mêmes (§2.1).

Variable	Obligatoire ?	Où la trouver	Intégration
`APP_MODE`	Obligatoire (`live`)	Vous	Base
`SESSION_SECRET`	Obligatoire	Vous (`openssl rand -base64 48`)	Base
`APP_BASE_URL`	Obligatoire	Votre domaine, `https://…` sans `/` final	Base, Sign-In, webhooks
`SYNC_POLL_SECONDS`	Optionnelle (30)	30 à 60	Base
`LOG_LEVEL`	Optionnelle (`info`)	`debug` / `info` / `warn` / `error` / `silent`	Journaux
`FOUNDER_THEO_EMAIL` , `FOUNDER_PABLO_EMAIL`	Obligatoires	Vos adresses Google	Sign-In
`FOUNDER_THEO_TIMEZONE` , `FOUNDER_PABLO_TIMEZONE`	Obligatoires	`Europe/Paris` , `Europe/Madrid`	Base
`GOOGLE_OAUTH_CLIENT_ID` , `GOOGLE_OAUTH_CLIENT_SECRET`	Obligatoires	Google Auth Platform › Clients	Sign-In, Drive, Gmail
`SHEETS_SPREADSHEET_ID`	Obligatoire	URL du Sheet	Google Sheets
`GOOGLE_SERVICE_ACCOUNT_KEY_BASE64`	Obligatoire	Clé JSON du compte de service, en base64	Sheets, Drive, Calendar, sauvegardes
`PRIVATE_SHEET_ID_THEO` , `PRIVATE_SHEET_ID_PABLO`	Optionnelles	URL du Sheet privé de chacun	Google Sheets (privé)
`WISE_API_TOKEN`	Optionnelle (oblig. pour Wise)	Wise › Paramètres › API tokens (Read only)	Wise
`WISE_PROFILE_ID`	Optionnelle	`GET /v2/profiles` (profil BUSINESS)	Wise
`WISE_SCA_PRIVATE_KEY_BASE64`	Optionnelle	Votre clé RSA (publique déposée chez Wise), en base64	Wise
`WISE_WEBHOOK_PUBLIC_KEY_BASE64`	Optionnelle	Clé publique de production des webhooks Wise (doc Wise), en base64	Wise
`STRIPE_API_KEY`	Optionnelle (oblig. pour Stripe)	Stripe › Développeurs › Clés API › clé restreinte	Stripe
`STRIPE_WEBHOOK_SECRET`	Optionnelle	Stripe › Webhooks › destination › secret `whsec_…`	Stripe
`WEBSITE_WEBHOOK_SECRET`	Optionnelle (oblig. pour les formulaires)	Vous (32+ car.), aussi côté site	Formulaires du site

Variable	Obligatoire ?	Où la trouver	Intégration
`INBOX_WEBHOOK_SECRET`	Optionnelle (oblig. pour le relais)	Vous, aussi dans Make/Zapier/n8n	Boîte À répondre
`META_APP_SECRET`	Optionnelle	App Meta › Paramètres › Général	Boîte (Instagram)
`META_VERIFY_TOKEN`	Optionnelle	Vous, aussi dans la config Webhooks Meta	Boîte (Instagram)
`INSTAGRAM_BUSINESS_ACCOUNT_ID`	Optionnelle (oblig. pour 1 compte)	Explorateur Graph API	Instagram
`INSTAGRAM_ACCESS_TOKEN`	Optionnelle (oblig. pour 1 compte)	Utilisateur système Meta (ou Instagram Login)	Instagram
`INSTAGRAM_ACCOUNT_KEY`	Optionnelle	`ig-valemont` (défaut) ou `ig-thirty`	Instagram
`INSTAGRAM_TOKEN_EXPIRES_AT`	Optionnelle	Date d'expiration d'un jeton 60 jours	Instagram
`INSTAGRAM_ACCOUNTS_JSON`	Optionnelle (à la place des 4 ci-dessus)	Vous (liste JSON, voir `.env.example`)	Instagram
`INSTAGRAM_API_BASE_URL`	Optionnelle	Seulement `https://graph.instagram.com/v21.0` en Instagram Login	Instagram
`SNAPSHOT_JOB_SECRET`, `SNAPSHOT_JOB_TOKEN`	Optionnelles	Vous (si planificateur autre que Vercel)	Tâche de capture
`TRACKER_WEBHOOK_URL_THEO` / `_PABLO`	Optionnelles	Ton app de suivi (https)	Mon tracker
`TRACKER_WEBHOOK_SECRET_THEO` / `_PABLO`	Optionnelles (oblig. avec l'URL)	Vous (16+ car.)	Mon tracker
`TRACKER_FEED_TOKEN_THEO` / `_PABLO`	Optionnelles	Vous (24+ car.)	Mon tracker
`TRACKER_DAILY_SUMMARY_THEO` / `_PABLO`	Optionnelles	`1` pour activer	Mon tracker
`BACKUP_DRIVE_FOLDER_ID`	Optionnelle (recommandée)	URL du dossier / Drive partagé de sauvegarde	Sauvegardes
`BACKUP_RETENTION_DAYS`	Optionnelle (30)	1 à 365	Sauvegardes
`BACKUP_JOB_SECRET`, `BACKUP_JOB_TOKEN`	Optionnelles	Vous (si planificateur autre que Vercel)	Sauvegardes
`CRON_SECRET`	Optionnelle (recommandée sur Vercel)	Vous (24+ car.)	Sauvegardes + capture

Variable	Obligatoire ?	Où la trouver	Intégration
`SHEETS_API_BASE_URL`, `GOOGLE_API_BASE_URL`, `WISE_API_BASE_URL`, `STRIPE_API_BASE_URL`, `DEMO_BACKEND`, `FAKE_SHEETS_URL`, `FAKE_SHEETS_SPREADSHEET_ID`, `DEMO_ACCESS_CODE`	**Jamais en production**	Tests et démo uniquement	—

Côté site (pas dans Vercel de Valemont OS) : `WEBSITE_WEBHOOK_SECRET` (même valeur) et `VALEMONT_WEBHOOK_URL`. Les identifiants d'agendas et du dossier Drive se saisissent **dans l'application** (Réglages › Calendriers, Réglages › Espace de travail), pas en variables.

---

4. Rappels de sécurité

- **Les secrets vivent uniquement dans Vercel** (cochés *Sensitive*) et dans votre gestionnaire de mots de passe. Jamais dans un chat, un mail, le Sheet, un ticket, une capture d'écran ou le code. L'application ne les affiche jamais : Réglages › Intégrations montre seulement « définie / manquante ». Les journaux masquent clés, e-mails et numéros.
- **Moindre privilège partout :** compte de service sans rôle (seulement les partages utiles), dossier Drive en Lecteur, agendas perso en « disponibilités seulement », Wise en **lecture seule**, Stripe en **clé restreinte Lecture**, Instagram en statistiques/lecture, Gmail en **envoi seulement**, Drive en `drive.file` (seulement les fichiers créés par l'app).
- **Ce que l'application ne fait jamais :**
  - envoyer un e-mail, un WhatsApp, une réponse Instagram ou LinkedIn sans **ton clic** ;
  - lire ta boîte Gmail ;
  - déplacer de l'argent, payer, rembourser (Wise et Stripe en lecture seule) ;
  - contacter un contact désinscrit (blocage côté serveur, pour vous deux) ;
  - montrer à l'un les données privées de l'autre (tâches privées, détails d'agenda perso) ;
  - utiliser la démo à la place des vraies données : sans identifiants, le mode LIVE affiche une erreur de configuration.
- **Rotation :**
  - fuite ou doute sur un secret → le régénérer chez le fournisseur **et** dans Vercel en même temps, redéployer ;
  - clé Google → nouvelle clé, variable, redéploiement, **puis** suppression de l'ancienne ;
  - `SESSION_SECRET` changé = tout le monde est déconnecté (c'est la façon de révoquer une session volée) ;
  - jeton Instagram 60 jours : l'application crée une tâche 14 jours avant l'expiration ;
  - départ d'un prestataire ou changement d'ordinateur : refaire les jetons qu'il a pu voir.
- **Un seul propriétaire par ressource, et c'est Valemont :** comptes Google, portefeuille Meta, compte Vercel, Wise, Stripe. Activez la double authentification partout.
- **Sauvegardes :** la copie JSON téléchargeable contient des données personnelles, gardez-la dans un endroit privé. Une restauration se fait toujours dans un **nouveau** Sheet, jamais par-dessus la production.

Délais à anticiper (récapitulatif)

Quoi	Délai	En attendant
Vérification Google pour `gmail.send` (app Externe en production)	plusieurs jours à quelques semaines	App Interne (Workspace), ou mode Test avec vos deux adresses, ou brouillon Gmail pré-rempli
App Review Meta (commentaires/messages Instagram)	quelques jours à quelques semaines	Relais Make « Watch comments » vers la boîte
API LinkedIn Community Management	long, non garanti	Exports .xls/.xlsx importés chaque semaine
Certificat du domaine Vercel	quelques minutes à quelques heures (DNS)	Adresse `*.vercel.app` pour tester (penser à mettre à jour `APP_BASE_URL` et les URI Google ensuite)